

Deep Reinforcement Learning for HPC Job Scheduling: A Statistical Evaluation

Justin M. Cheney

University of the Western Cape

Robert Sobukwe Rd, Bellville,

Cape Town, 7535

South Africa

4323819@myuwc.ac.za

ABSTRACT

Supercomputers and their workloads have grown steadily more heterogeneous, and job scheduling has moved from static heuristics to Deep Reinforcement Learning (DRL) policies, though no consensus exists on which algorithm suits the task. Comparisons to date, however, benchmark DRL policies only against heuristics, leaving open a prior question: how much of the reported performance comes from the learned policy, and how much from the action masking that makes such policies runnable at all. This paper addresses that question by comparing six DRL treatments (PPO, A2C and DQN, each with and without action masking) against three Slurm heuristics and, critically, a seed-paired masked uniform-random control, across ten seeds on homogeneous and heterogeneous Slurm traces under a non-parametric, rank-based framework. On the homogeneous trace no DRL treatment outperforms the random control, and results against the heuristics are mixed, so the measured performance is supplied by the mask rather than by anything learned. On the heterogeneous trace MaskablePPO holds parity on average waiting time and improves maximum slowdown by up to 22%, though no treatment separates from the others at $n = 10$. MaskablePPO is the best-ranked treatment that completes an evaluation episode on both traces; however, the differences between algorithms are not significant in this data. Masking therefore delivers deployability rather than schedule quality, and a masked random control is recommended as a baseline for this literature.

KEYWORDS

High-Performance Computing, Deep Reinforcement Learning, Statistical Analysis, Job Scheduling

1 INTRODUCTION

High-Performance Computing (HPC) systems consist of several software components including package managers, job schedulers, and monitoring tools [8]. Job schedulers manage job queue ordering and handle resource allocation, determining when jobs execute and which computational resources they have access to [24]. As HPC workloads have diversified (from traditional, purely arithmetic-based tasks to machine learning and data-centric tasks) scheduling algorithms have had to evolve to address the increased complexity [31]. Job scheduling has evolved from the simple heuristics (First-Come-First-Served, Shortest Job First) to meta-heuristic optimisation techniques (genetic algorithms, particle swarm optimisation) [24]. As heterogeneous clusters became more prevalent, a new set of challenges were introduced [31].

Traditional heuristics struggle to address multiple objectives across diverse hardware [24]. Deep Reinforcement Learning (DRL)

models address this by enabling schedulers to learn adaptive policies from previous behaviour [6, 34]. However, the rapid adoption of DRL raises a question: which algorithm family (policy gradient, value-based, actor-critic) has the best performance for modern HPC workloads? An algorithm that reduces the makespan may fall short in fully utilising the system's resources, therefore justifying a multi-objective evaluation. Previous studies have compared DRL schedulers against traditional methods [25, 27]. However, systematic statistical evaluation across multiple DRL algorithm families has not been explored. A structured reading of 53 recent papers finds PPO-family methods in 40% of them, roughly double the value-based share, while none evaluates the policy-gradient, value-based and actor-critic families together. The field has converged on a default without establishing that the default is correct.

This paper addresses that gap through a controlled comparison of six DRL algorithms: MaskablePPO, MaskableDQN and MaskableA2C, with their unmasked counterparts, spanning the policy gradient, value-based and actor-critic families. All six are trained and evaluated on real Slurm traces of approximately 84,000 jobs under identical conditions, across waiting time, slowdown, turnaround time and resource utilisation, using Friedman tests, Nemenyi post-hoc analysis and Wilcoxon-based confidence analysis [1]. Implementations from stable-baselines3 and sb3-contrib [23] standardise the comparison, with additional code introduced only where the environment requires it. The focus is on physical HPC clusters rather than cloud systems, where elastic resource pools change the problem in kind. The investigation is guided by three research questions:

1. What does action masking contribute to an algorithm's schedule quality and deployability, and how much of the measured performance is able to be attributed to the mask rather than learning?
2. What is the justification for the field's concentration on PPO and its variants, and on which axis of cost or quality?
3. Under what workload conditions is it worth training a DRL scheduler given their inference costs in comparison to using a traditional heuristic?

This paper makes the following contributions:

1. A systematic, head-to-head statistical comparison of the three major DRL algorithm families for HPC job scheduling, under identical workloads, reward structures and training budgets.
2. An isolation of the effect of action masking, pairing each masked algorithm with its unmasked counterpart, separating gains from constraint enforcement from gains from policy learning.
3. A non-parametric statistical protocol combining Friedman, Nemenyi and Wilcoxon-based analysis, suited to the non-normal, seed-dependent performance distributions typical of DRL.

4. A multi-objective evaluation across four metrics, exposing trade-offs that single-metric studies conceal.
5. An empirical test of the assumed dominance of PPO in HPC scheduling, a claim widely cited but rarely validated in this domain.
6. Practical guidance on when training a DRL scheduler is justified relative to established heuristics, based on real production traces rather than synthetic benchmarks.

The remainder of this paper reviews the literature and the gaps motivating the study (Section 2), sets out the methodology (Section 3), presents the results (Section 4), discusses their implications and limitations (Section 5) and concludes (Section 6).

2 LITERATURE REVIEW

HPC job scheduling has evolved from rule-based heuristics to learning-based adaptive techniques. This section reviews that progression and identifies the methodological gaps motivating the present study.

2.1 HPC Job Scheduling

Job schedulers fulfil two primary functions: queue management, which determines execution order under a priority policy, and resource allocation, which assigns cores, memory, GPUs and storage while preventing contention [34]. Effective scheduling maximises utilisation, minimises waiting times and ensures fairness across users [24]. These objectives are not independent. Early systems prioritised throughput and makespan, whereas modern deployments must balance performance, cost, energy consumption and fairness simultaneously [24]. That shift is why scheduler evaluation cannot rest on a single metric.

2.2 Traditional and Meta-heuristic Techniques

Traditional techniques use static heuristics. First-Come-First-Served (FCFS) is simple and fair, but large jobs block the queue and leave resources idle even when smaller jobs could run, a condition known as head-of-line blocking [28]. Shortest Job First (SJF) lowers average waiting but starves long jobs and depends on duration estimates production workloads rarely supply. Backfilling recovers part of that loss by filling gaps between larger reservations [15], and schedulers such as Slurm [36] add quotas, ageing and fair-share policies [10]. These methods are predictable and cheap, hence their continued production use, but they struggle with the resource requirements and dynamic workloads of modern heterogeneous systems [34].

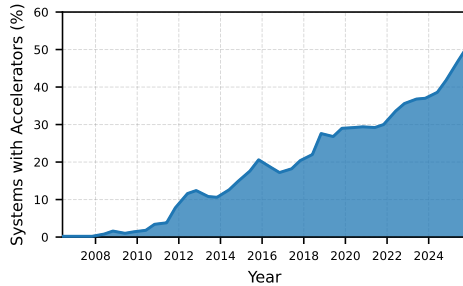


Figure 1: Growth of heterogeneous systems in the TOP500 supercomputers (2006–2025) [31]

Meta-heuristics search larger solution spaces through bio-inspired strategies such as genetic algorithms, particle swarm optimisation and simulated annealing. They outperform simple heuristics

but remain computationally expensive, sensitive to parameter tuning and, most importantly, optimise each workload instance independently: they cannot generalise a policy to changing workload patterns without re-optimisation [24]. That limitation is structural, and it motivates the shift to learning-based methods.

2.3 Deep Reinforcement Learning for Job Scheduling

Reinforcement learning addresses the generalisation limitation through trial-and-error policy optimisation, requiring no labelled optimal decisions, which are unavailable in practice. DRL extends this to the high-dimensional state spaces of large job queues and heterogeneous resources through neural network function approximation, with scheduling formalised as a Markov Decision Process over the queue and cluster state [9, 30]. DRL has been widely explored for cloud and edge scheduling using value-based methods (DQN variants), policy gradient approaches (PPO, TRPO) and actor-critic techniques (A3C, A2C). Recent work concentrates heavily on one: PPO-family methods appear in 40% of the 53 papers surveyed and in 45% of the 60 systems they present. Schulman *et al.* [26] attribute this to superior stability and reliability, but the claim originates in continuous control benchmarks and has not been validated within HPC scheduling. The concentration is therefore better read as inherited convention than as an empirical finding, and it constitutes the first gap addressed here.

2.3.1 Algorithm Families

Job scheduling is fully observable and is modelled as an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$ [30], in which the agent observes state s_t , selects action a_t under policy π , receives reward r_t and transitions to s_{t+1} . The objective is the policy maximising expected discounted return:

$$\sum_{t=0}^{\infty} \gamma^t r_t$$

Value-based methods approximate the optimal action-value function via the Bellman optimality equation, Deep Q-Networks applying this with experience replay and target networks [21]. Policy-based methods optimise the policy directly by gradient ascent on expected return, PPO stabilising this with a clipped surrogate objective [26]. Actor-critic methods combine the two, the critic reducing variance in the policy gradient; A2C is the synchronous variant of A3C [20]. Each family assumes different things about sample efficiency, exploration and stability, so a comparison confined to one cannot speak to the others.

2.3.2 Action Masking

Action masking simplifies the agent’s decision process by excluding infeasible actions [29]. For value-based methods, invalid actions are masked before arg max selection; for policy-based methods, invalid probabilities are zeroed before normalisation. In HPC scheduling this prevents selecting jobs whose resource demands exceed current capacity. Masking is widely adopted, but typically enabled by default and reported as part of the environment rather than as an experimental variable, so published figures conflate the benefit of constraint enforcement with that of learning. The independent contribution of masking is therefore unmeasured, which is the second gap. This paper compares masked variants against their unmasked counterparts to quantify it.

2.3.3 DRL Applications in Job Scheduling

DRL has been adopted across cloud computing [7, 16], edge systems [32] and HPC clusters, targeting makespan, utilisation and fairness. The formulation used here originates with DeepRM [18], which cast packing-style resource management as a policy-gradient learning problem, and Decima [19] is the standard cited counterpoint for graph-structured cluster scheduling. Wu *et al.* [35] employed Double DQN for dual-objective optimisation but compared only against three meta-heuristics, leaving its standing relative to other DRL families unknown. Policy gradient schedulers dominate the remainder: Gao *et al.* [7] applied Gated GtrXL with PPO, and Wang *et al.* [34] proposed hierarchical PPO for two-level scheduling. Both cite the stability and sample efficiency of PPO as motivation, though neither offers direct algorithmic comparison. The reference point for the problem is RLScheduler [37], which selects a policy-gradient actor-critic on the argument that the variance of batch scheduling would destabilise a value-based agent, and validates it against FCFS, SJF, WFP3, UNICEP and F1 but against no unlearned policy, so both assumptions this paper tests are ones its closest predecessor asserts rather than measures. Others extend the base algorithms through multi-agent formulations [17] and transfer learning [33].

These studies demonstrate the potential of DRL for adaptive scheduling, but their evaluations differ markedly in statistical rigour, performance metrics, workload characteristics and choice of baseline, which makes their results incommensurable: an improvement over a meta-heuristic on a synthetic workload cannot be compared with an improvement over Slurm on a production trace. This heterogeneity is the third gap.

2.4 Limitations and Research Gaps

The benefits of DRL come with limitations bearing on deployability. Training demands large datasets and specialised hardware, and state and action spaces expand rapidly with workload complexity. Sparse rewards limit learning for long-running objectives such as makespan or utilisation, where the outcome is visible only once all jobs complete. The approach is also a black box, and in mission-critical environments the inability to explain a decision restricts adoption. These costs make the choice of algorithm a practical matter rather than an academic one, yet they are rarely weighed against the gains reported.

3 METHODOLOGY

This section presents the workflow and statistical analysis for an empirical comparison of ten treatments. Six are DRL algorithms: PPO, A2C, DQN and MaskablePPO from stable-baselines3 and sb3-contrib [23], together with custom MaskableA2C and MaskableDQN, so that each base algorithm appears both with and without masking and masking is treated as a regularised experimental factor. Three heuristics, SJF, Last-Come-First-Served (LCFS) and Universal Cobalt Evaluation Priority (UNICEP), act as the traditional comparison, and the tenth treatment is a masked uniform-random control. All ten are compared across two Slurm workload traces, using HPCSim [34] as the environment under fixed seeds for reproducibility.

3.1 Environment and Dataset

HPCSim [34] provides a Gymnasium-based scheduling environment where the observation space comprises the cluster

state and a job queue window, and the action space represents job selection decisions. Table 1 gives the configuration applied uniformly across all experiments. The “best_fit” allocator is fixed as a controlled variable: a topology-aware allocator only helps the deep learning workload, so fixing it keeps allocation consistent across both workload types and ensures comparisons reflect scheduling policy rather than allocation strategy. Backfill is removed from the baselines for the same reason. It is present in production Slurm, but allowing it would measure the availability of an action the DRL agents cannot take rather than policy quality, and separating mechanism from policy is what the third research question asks.

The dataset is taken from Wang *et al.* [34]: two Slurm traces, one homogeneous (physical), the other heterogeneous (deeplearn). A 70/30 dev-holdout split was applied to both, ordered by submission time, which prevents data leakage and reflects realistic conditions where future arrivals are unknown. The holdout splits are not accessed during the run and are reserved for the final generalisation assessment.

Table 1: Experimental configuration: environment, hyperparameters and dataset

Parameter	Value	Notes
<i>HPCSim environment</i>		
Allocator	best_fit	Fixed as controlled factor
Window Size	512	Observable queue depth
Tail Size	64	Queue tail buffer
Random jobs	False	Deterministic trace replay
Backfill	False	Uniform across algorithms
<i>Hyperparameters</i>		
Hidden Layers	[4096, 2048, 1024]	All
Activation function	tanh	All
Discount factor (γ)	0.99	All
Policy	MultiInputPolicy	All
Entropy Coefficient	0.0	All
Replay Buffer Size	150,000	DQN variants only
<i>Dataset: physical / deeplearn</i>		
Trace	physical_jobs.csv / deeplearn_jobs.csv	
Topology	physical/deeplearn_topology.txt	Cluster layout
Node file	nodes.csv	Node resource definitions
Total Jobs	84,135 / 68,720	
Train Jobs (70%)	58,894 / 48,104	
Test Jobs (30%)	25,241 / 20,616	
Temporal Range	37d 11h 42m / 382d 5h 21m	

Table 2: Performance Metrics

Metric	Definition	Direction	Role
avg_waiting	Mean job wait time	Lower	Primary
avg_slowdown	Mean bounded slowdown	Lower	Primary
avg_turnaround	Mean turnaround time	Lower	Secondary
max_waiting	Maximum wait time	Lower	Secondary
max_slowdown	Maximum bounded slowdown	Lower	Secondary
cpu_utilization	CPU usage ratio	Higher	Secondary
gpu_utilization	GPU usage ratio	Higher	Secondary
decision_latency	mean ms per decision	Lower	Operational
eval_wall_s	evaluation wall-clock seconds	Lower	Operational
episode_reward	shaped return	Higher	Operational

3.2 Training and Evaluation

All agents share the architecture in Table 1, with the replay buffer specific to the DQN variants. Ten static seeds fix random, numpy and torch. Ten seeds per algorithm on one split gives 60 runs, and 120 across both splits.

All 120 trained models are evaluated deterministically on their respective dev splits under their own seed, collecting the primary, secondary and operational metrics used by [34] and defined in Table 2. Bounded slowdown is computed as $\max\left(1, \frac{w+r}{\max(r, \tau)}\right)$ with the run-time floor $\tau = 10$ s, the convention HPCTSim applies; the bound matters because the scheduling literature uses several. Per-run metrics and metadata are written to “run_log.csv” with an accompanying json file, and aggregated across the ten seeds per algorithm. The reward throughout training is the negative mean queued waiting time in hours ($R = -\frac{\text{avg_waiting}}{3600.0}$).

3.3 Statistical Framework

Adapting the framework of Carrasco *et al.* [1], a non-parametric repeated-measures design is used, with seeds as blocks and treatments as conditions. At $n = 10$ normality cannot be assumed, confirmed per treatment by Shapiro-Wilk, so rank-based methods apply. Friedman is the omnibus test per metric per trace ($\alpha = 0.05$), with Kendall’s $W = \chi_w^2 / N(k - 1)$ reported alongside every p-value and interpreted following [13], to avoid sample-size-driven conclusions. Where Friedman is significant, Nemenyi post-hoc comparisons follow, with Critical Difference plots showing non-significant rank groupings [2]. Wilcoxon confidence intervals and confidence curves give effect-size context, and convergence uses the Page trend test [3].

The blocking unit is a seed, not a dataset, and this bounds what the omnibus test can claim. The framework being adapted blocks on datasets, because the null it tests is that algorithms rank equivalently across independent problems. Blocking on seeds within two traces instead tests whether treatments rank consistently under re-initialisation, which is a variance question rather than a generality question. The Nemenyi critical difference reported below should be read accordingly: it does not license a claim that these rankings transfer to unseen workloads, and the two traces here cannot support one.

A single best algorithm is Pareto-selected across the primary and secondary metrics, ties broken on avg_waiting $\rightarrow$ avg_slowdown $\rightarrow$ cpu_utilization, then tested against each heuristic with two one-sided tests (TOST) [12], since a non-significant difference alone cannot prove equivalence against a stated margin. The $\pm 10\%$ equivalence margin is set to the smallest

difference in average waiting an operator could act on at this cluster scale, roughly three minutes on the physical trace, rather than derived from the data. Being deterministic single runs, the heuristics take the one-sample Wilcoxon signed-rank test with a Hodges-Lehmann estimate [14]. P-values are Holm-corrected [11] within each comparison family, though not across metric families, and the intervals on the bar figures are bootstrap intervals on the mean [5].

A random action-selection control is included as a reference condition, drawing uniformly at each decision over the valid actions of the same MDP, under the same mask, without backfill, on the same ten seeds. Being paired by seed, it is compared with a two-sided Wilcoxon signed-rank test, a stronger design than the one-sample form the deterministic heuristics permit. It establishes what an unlearned policy achieves on each trace, without which a learned policy matching a heuristic cannot be distinguished from a trace on which any policy performs comparably. Finally, all algorithms are evaluated on the holdout splits across the same seeds to test generalisation.

3.4 Reproducibility and Workflow

A triple-layer stack provides reproducibility: Git tracks source history with commit hashes and sha256 manifest fingerprints in all output metadata, a Nix flake pinned to nixos-25.05 [4] makes every dependency content-addressed rather than version-pinned, and Apptainer containerises the environment for cluster execution. Snakemake 9.4.3 [22] orchestrates the pipeline, with config-driven separation of smoke test and production runs.

4 RESULTS

This section reports results following the methodology in Section 3, presenting the physical and deeplearn traces in turn before examining generalisation, training and inference cost, and training diagnostics. Except where the generalisation subsection states otherwise, every table and figure below reports dev-split performance, that is, performance on the 70% split the models were trained on; the holdout results are reported separately in Section 4.3. Shapiro-Wilk rejects normality across both traces at $n = 10$: on the physical trace A2C gives ($W = 0.634, p = 0.00014$) and DQN ($W = 0.753, p = 0.0039$) on avg_waiting, with the deeplearn trace stronger again at $p \sim 10^{-7}$. This justifies the rank-based tests set out in Section 3. Those tests and the percentage deltas are computed on per-seed medians, with means used only to indicate magnitude, so a mean far above the median signals

Table 3: Physical trace - performance metrics (mean $\pm$ std.)

Algorithm	Avg Wait (s)	Avg Slowdown	Avg Turnaround (s)	Max Wait (s)	Max Slowdown	CPU Util
MaskablePPO	2,243 $\pm$ 41	7.49 $\pm$ 0.34	10,985 $\pm$ 41	69,126 $\pm$ 18,255	2,918 $\pm$ 44	0.1057 $\pm$ 0.0000
MaskableDQN	4,474 $\pm$ 1,684	116.39 $\pm$ 85.42	13,216 $\pm$ 1,684	2,065,995 $\pm$ 1,309,114	193,701 $\pm$ 122,640	0.0982 $\pm$ 0.0070
MaskableA2C	2,223 $\pm$ 42	7.36 $\pm$ 0.14	10,965 $\pm$ 42	49,881 $\pm$ 17,225	2,966 $\pm$ 67	0.1057 $\pm$ 0.0000
PPO†	8,470 $\pm$ 6,177	125.33 $\pm$ 120.52	17,065 $\pm$ 6,159	47,693 $\pm$ 13,031	4,060 $\pm$ 949	0.0974 $\pm$ 0.0029
DQN†	6,229 $\pm$ 4,478	87.23 $\pm$ 84.90	14,820 $\pm$ 4,495	54,745 $\pm$ 40,343	5,325 $\pm$ 4,034	0.0980 $\pm$ 0.0027
A2C†	5,500 $\pm$ 4,816	63.99 $\pm$ 92.35	14,094 $\pm$ 4,803	34,767 $\pm$ 6,897	3,357 $\pm$ 531	0.0968 $\pm$ 0.0006
LCFS	2,090	5.88	10,832	58,418	3,096	0.1060
SJF	2,176	8.14	10,918	69,909	3,060	0.1051
UNICEP	2,499	15.87	11,241	63,851	3,513	0.1053
Random (masked)	2,166 $\pm$ 76	6.01 $\pm$ 0.19	10,908 $\pm$ 76	53,819 $\pm$ 18,028	2,599 $\pm$ 175	0.1058 $\pm$ 0.0001

divergence on a minority of seeds rather than a uniformly poor policy.

Two conventions apply throughout. The first is truncation. None of the three unmasked treatments completed a full evaluation episode on either trace. Without a mask the policy can select an action naming no allocatable job, which HPCSim treats as a request to advance time; once the event queue is empty, simulated time freezes on an identical state and the episode cannot terminate. A hang guard stops the run after 500,000 consecutive steps without a job completion, reached at 591,200–591,400 decisions on physical and 620,100–620,300 on deeplearn, identically across all ten seeds. Those results are therefore biased, and the treatments are excluded from best-value marking and marked “†” throughout. The second is the backfill band. Each heuristic was evaluated with backfill both enabled and disabled, and the width of that band is reported per trace below; the baseline figures quoted elsewhere are the no-backfill ones, for the reason given in Section 3.

4.1 Homogeneous Workload: Physical Trace

Truncation leaves $\approx 91k$ productive steps on this trace, identically across all ten seeds. Backfill is worth up to 12.3% of average waiting to the heuristics here (UNICEP), but it does not change the outcome: the heuristics outperform the DRL treatments either way. Due to homogeneity in the trace, gpu_utilization is 0.000 for all treatments and is omitted from this subsection.

Table 3 splits on Avg Wait (s): MaskableA2C and MaskablePPO sit within a few percent of the heuristics at 2,223 s and 2,243 s while every unmasked treatment is 2.5–4 $\times$ worse, which is what happens when a policy spends its episode on invalid actions that earn negative rewards without placing a job. MaskableDQN instead diverges, plausibly from too small a buffer size to optimise Q-values. The Friedman test rejects the null for all nine metrics tested on this trace, with χ^2 from 22.11 to 47.14 and Kendall’s W from 0.442 to 0.943, agreement being highest on the operational metrics where the masked/unmasked difference is mechanical. One caution: cpu_utilization is significant ($p = 2.4e-05$, $W = 0.579$) only because rank tests are scale-free. The masked treatments differ by under 0.1%, so it should carry no practical claim.

Table 4 gives the top two mean ranks to MaskablePPO (1.7) and MaskableA2C (1.9), with the Nemenyi post-hoc returning $p = 1.000$ between them, so they cannot be ordered at $n = 10$, $k = 6$. The Page trend test shows average waiting improving monotonically along the masking-ordered sequence ($L = 838$, $p = 6e-6$, decreasing), and that trend holds for all primary and secondary metrics bar maximum waiting ($p = 1.0000$); because the unmasked arm never terminates an episode, this trend is reported

Table 4: Nemenyi post-hoc Ranking - Avg Waiting Time

Algorithm	Mean Rank	Median	Mean	p vs. best
MaskablePPO	1.7	2,242	2,243	---
MaskableA2C	1.9	2,212	2,223	1.000
A2C†	4.1	2,923	5,500	0.047*
MaskableDQN	4.4	5,008	4,474	0.016*
PPO†	4.4	8,623	8,470	0.016*
DQN†	4.5	4,429	6,229	0.011*

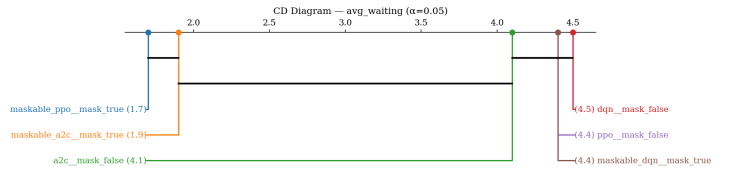


Figure 2: CD Diagram - Avg. Waiting Time (reduced)

as a descriptive ordering and is not read as a quantitative estimate of what masking contributes. Exact Wilcoxon intervals verify the tie against MaskablePPO: MaskableA2C’s $[-65.5, +13.3]$ s lies entirely inside the $\pm 10\%$ margin, so the two are formally equivalent, a stronger statement than the Nemenyi null of non-separation. The confidence curve agrees, peaking near zero and staying above α across a narrow range, the signature of an underpowered but tightly bounded comparison. With neither test separating them, the pareto selection returns MaskableA2C on the tie-break.

Those comparisons cover only the six DRL treatments, so the winning masked treatments must also be set against the heuristics and the random control (Figure 3). Deltas fall in $\approx [-20\%, +10\%]$, the outliers being LCFS fairness-to-short-jobs at +24.7% and +24.2% on average slowdown, and average slowdown against UNICEP at $> -53\%$ for both treatments; the corresponding max-slowdown figures are -17.0% and -15.7% .

In the case of the random control both DRL and the control are paired by seed, this also for a more equivalent comparison than that of the deterministic heuristics. In Table 5, results show that of the nine comparisons, seven separate and of those seven all are in favour of the random control. Though these results more likely point towards a fact that the algorithms were not able to effectively learn the schedule rather than that a random policy would be more effective in this context.

In this paper headroom is defined as the gap between the best heuristic and the random control. For the homogeneous workload of the physical trace this headroom in avg waiting is between LCFS

Table 5: Random Control - Primary Metrics

Treatment	Metric	Random median	DRL median	$\Delta\%$	Seeds better	p (Holm)	vs. random
MaskablePPO	Avg Wait	2,171.033	2,242.491	+4.3%	2/10	0.156	Not sep.
MaskablePPO	Avg Slow.	5.977	7.335	+24.8%	0/10	0.012	Worse
MaskablePPO	Max Slow.	2,570.333	2,914.091	+12.9%	1/10	0.029	Worse
MaskableDQN	Avg Wait	2,171.033	5,007.956	+139.9%	1/10	0.012	Worse
MaskableDQN	Avg Slow.	5.977	125.576	+1982.5%	0/10	0.012	Worse
MaskableDQN	Max Slow.	2,570.333	251,871.750	+9392.2%	0/10	0.012	Worse
MaskableA2C	Avg Wait	2,171.033	2,212.443	+3.1%	3/10	0.336	Not sep.
MaskableA2C	Avg Slow.	5.977	7.305	+22.7%	0/10	0.012	Worse
MaskableA2C	Max Slow.	2,570.333	2,962.182	+14.6%	0/10	0.012	Worse

Table 6: Deeplearn trace - performance metrics (mean $\pm$ std. dev.)

Algorithm	Avg Wait (s)	Avg Slowdown	Avg Turnaround (s)	Max Wait (s)	Max Slowdown	CPU Util	GPU Util
MaskablePPO	846 $\pm$ 9	3.64 $\pm$ 0.03	11,413 $\pm$ 9	31,442 $\pm$ 153	2,261 $\pm$ 58	0.1172 $\pm$ 0.0000	0.2612 $\pm$ 0.0000
MaskableDQN	45,729 $\pm$ 40,335	853.29 $\pm$ 810.96	56,296 $\pm$ 40,335	19,003,961 $\pm$ 12,150,302	1,182,525 $\pm$ 1,035,356	0.1095 $\pm$ 0.0055	0.2440 $\pm$ 0.0123
MaskableA2C	24,520,863 $\pm$ 77,476,701	670,447.13 $\pm$ 2,118,679.67	24,531,430 $\pm$ 77,476,701	51,687,957 $\pm$ 161,407,893	5,128,953 $\pm$ 16,058,567	0.1060 $\pm$ 0.0349	0.2361 $\pm$ 0.0778
PPO†	818 $\pm$ 9	3.66 $\pm$ 0.01	11,241 $\pm$ 25	30,684 $\pm$ 0	2,291 $\pm$ 0	0.1158 $\pm$ 0.0002	0.2559 $\pm$ 0.0009
DQN†	19,328 $\pm$ 51,265	446.45 $\pm$ 1,125.33	29,736 $\pm$ 51,262	5,263,952 $\pm$ 10,301,905	284,036 $\pm$ 761,632	0.1152 $\pm$ 0.0017	0.2542 $\pm$ 0.0036
A2C†	4,274 $\pm$ 10,923	66.55 $\pm$ 198.87	14,706 $\pm$ 10,970	521,773 $\pm$ 1,552,986	41,571 $\pm$ 123,972	0.1154 $\pm$ 0.0009	0.2550 $\pm$ 0.0014
LCFS	912	3.75	11,479	32,083	2,907	0.1172	0.2612
SJF	851	3.95	11,418	31,513	2,291	0.1172	0.2612
UNICEP	846	3.76	11,413	31,513	2,291	0.1172	0.2612
Random (masked)	1,170 $\pm$ 26	7.23 $\pm$ 0.29	11,737 $\pm$ 26	678,917 $\pm$ 237,272	7,004 $\pm$ 3,476	0.1172 $\pm$ 0.0000	0.2612 $\pm$ 0.0000

(2,090 s) and the random control (2,166 s) which is 3.7% wide. The results have shown that in the context of a single-resource trace the traditional scheduler is the defensible choice. This is due to the fact that the parity the masked treatments achieve is supplied by the mask and the low headroom rather than by anything learned. The other factors that may have improved the DRL treatments would have been a better reward proxy and optimisation-budget over SB3's default values.

4.2 Heterogeneous Workload: Deeplearn Trace

Truncation leaves $\approx 120k$ productive steps on this trace, again identically across all ten seeds. Backfill is worth up to 5.7% of average waiting to the heuristics (LCFS); inside that band SJF

(842 s) and UNICEP (844 s) edge ahead of MaskablePPO on average waiting, so the parity reported below is stated against the no-backfill baselines for the reason given in Section 3. The heterogeneous workload otherwise tells a different story, with the DRL agents holding the lead where the heuristics dominated on physical. Headroom explains most of it. Table 6 compares the best heuristic UNICEP (846 s) against a random control of 1,170 s, a 38.3% gap; against the physical trace's 3.7% that is a delta of 34.6%, meaning more space for a learned policy to show a benefit.

The omnibus results reject the null on all ten metrics at $W = 0.487-0.905$, with two caveats. GPU utilisation, significant at $\chi^2 = 28.63$ with $W = 0.573$, is driven entirely by divergence: utilisation is computed as $\frac{\text{resource-seconds}}{\text{total resource} \times \text{makespan}}$, so a diverging treatment such as MaskableA2C or MaskableDQN mechanically reports lower utilisation. Evaluation time is likewise skewed by A2C, DQN and PPO.

Table 7 is topped by unmasked PPO, but the truncated treatments are not eligible for selection, so the table is oriented on MaskablePPO as the winner. No other treatment separates from it (PPO† 0.066, A2C† 0.261, DQN† 0.891, MaskableA2C 0.999, MaskableDQN 0.324). The detail the ranking hides is MaskableA2C's median against its mean: it diverged on only 2 of 10 seeds, and on the median it actually outperforms MaskablePPO (837 s vs. 851 s). It is capable of the better result but not reliably.

MaskablePPO beats the random control outright on all primary metrics (Table 8), but against the heuristics it is a margins game. It matches UNICEP everywhere; the advantage is in the edge cases, beating LCFS and the random control on max slowdown by -22.0% and -63.2% . It does not schedule the average job better, it handles the awkward ones better, and those are more common in

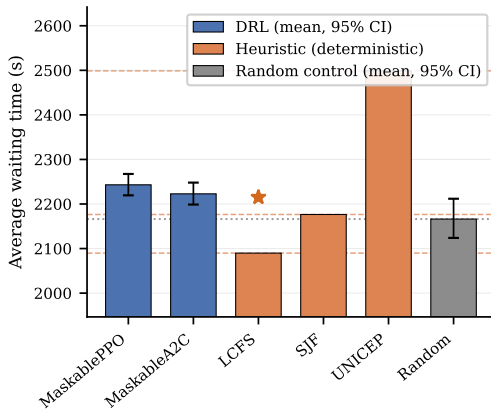


Figure 3: Bar Graph - Avg Waiting time (reduced)

Table 8: Random Control Comparison - Primary metrics

Treatment	Metric	Random median	DRL median	$\Delta\%$	Seeds better	p (Holm)	vs. random
MaskablePPO	Avg Wait	1,171.719	850.901	-28.0%	10/10	0.012	Better
MaskablePPO	Avg Slow.	7.199	3.645	-49.8%	10/10	0.012	Better
MaskablePPO	Max Slow.	5,921.342	2,267.550	-63.2%	10/10	0.012	Better
MaskableDQN	Avg Wait	1,171.719	45,551.950	+3900.3%	1/10	0.023	Worse
MaskableDQN	Avg Slow.	7.199	721.107	+9995.8%	1/10	0.023	Worse
MaskableDQN	Max Slow.	5,921.342	957,643.358	+12070.5%	1/10	0.023	Worse
MaskableA2C	Avg Wait	1,171.719	837.357	-27.8%	8/10	1.000	Not sep.
MaskableA2C	Avg Slow.	7.199	3.646	-48.6%	8/10	1.000	Not sep.
MaskableA2C	Max Slow.	5,921.342	2,307.000	-55.7%	8/10	1.000	Not sep.

Table 7: Nemenyi Post-hoc Ranking - Avg. Waiting Time

Algorithm	Mean Rank	Median	Mean	p vs. best
PPO†	1.7	821	818	0.066
A2C†	2.2	821	4,274	0.261
DQN†	3.1	825	19,328	0.891
MaskablePPO	4.0	851	846	---
MaskableA2C	4.3	837	24,520,863	0.999
MaskableDQN	5.7	45,552	45,729	0.324

heterogeneous workloads. Average waiting still sits within 1% of the heuristics (SJF 851 s, UNICEP 846 s), so the training costs in Section 4.4 decide whether this is viable. Cross-trace comparison

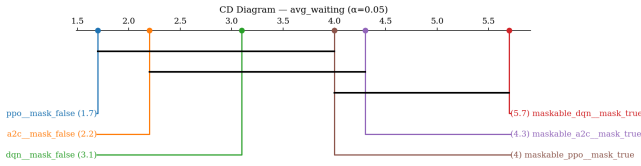


Figure 4: CD Diagram - Average Waiting Time (reduced)

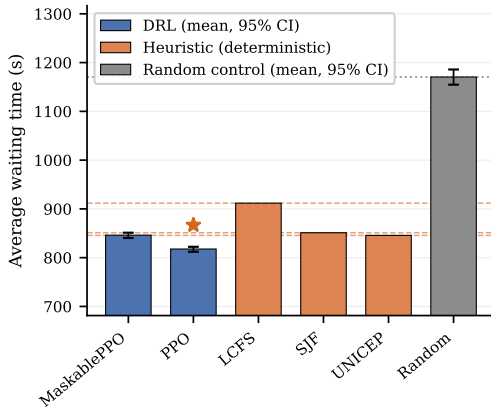


Figure 5: Bar Graph - Average Waiting Time (reduced), Bootstrap 95% CIs

Table 9: Physical Trace - Training & Inference Cost

Algorithm	Train (h)	Runs	Eval (s)	Eval Ratio	Latency (ms)
MaskablePPO	3.81 $\pm$ 0.09	10	2,677	35×	22.31
MaskableDQN	5.76 $\pm$ 0.68	10	1,434	19×	2.77
MaskableA2C	4.34 $\pm$ 0.17	10	2,612	34×	22.55
PPO†	3.42 $\pm$ 0.24	10	19,842	257×	23.91
DQN†	5.63 $\pm$ 0.66	10	6,297	81×	2.68
A2C†	3.77 $\pm$ 0.12	10	18,954	245×	20.94

also needs care: deeplearn is more lightly loaded at 30 nodes with 120 GPUs against physical's 87 nodes and 6,264 cores, and two traces support the hypothesis that DRL performs better in multi-resource environments only as an observed direction, not an established result.

4.3 Generalisation

Each treatment was then evaluated on the holdout split it had never seen. The splits are not directly comparable, physical showing a $\sim 4\times$ decrease in average waiting and deeplearn a $\sim 3\times$ rise, but the Friedman and Nemenyi ordering is largely static: MaskablePPO and MaskableA2C stay 1st and 2nd on physical, and on deeplearn PPO and MaskablePPO swap while holding the first two slots, which verifies the winners were not manufactured. Lower down, physical shifts more, with MaskableDQN moving to 6th, PPO to 4th and A2C to 3rd, pointing to the DQN resource constraints in Section 4.5.

4.4 Training and Inference Cost

Evaluation time is inflated for the truncated treatments by the 500k stall steps, though masking still visibly cuts it because unmasked policies spend most of their time on invalid actions. Training time and decision latency are unaffected, and Table 9 and Table 10 agree on both: the A2C and PPO families train faster than the DQN family on both workloads (3.42–4.34 h and 1.23–1.61 h against 5.63–5.76 h and 1.71–1.73 h), while the DQN family has the best per-decision latency, since taking a max Q-value is cheaper than sampling from a distribution.

That latency comes with lower stability, partly a hardware constraint. The per-job and per-node observation encoding costs ~ 276 KB per stored transition, so the 150k buffer consumes ~ 44 GB of a 128 GB node and the desired 1M buffer would have needed ~ 270 GB. The penalty falls only on DQN, because off-policy replay is structurally expensive here, and the smaller buffer compounds: at `train_freq = 4`, `gradient_steps = 1` and `batch_size = 2048` the family performs $\sim 750,000$ updates over 3M timesteps, replaying $\sim$

Table 10: Deeplearn trace - Training & Inference Cost

Algorithm	Train (h)	Runs	Eval (s)	Eval Ratio	Latency (ms)
MaskablePPO	1.23 ± 0.02	10	629	13×	4.97
MaskableDQN	1.73 ± 0.02	10	352	7×	0.67
MaskableA2C	1.45 ± 0.27	10	1,202	25×	5.06
PPO†	1.26 ± 0.25	10	4,542	94×	4.74
DQN†	1.71 ± 0.02	10	2,509	52×	1.08
A2C†	1.61 ± 0.29	10	5,022	104×	4.89

512 stored samples per environment sample, the most likely source of the divergence.

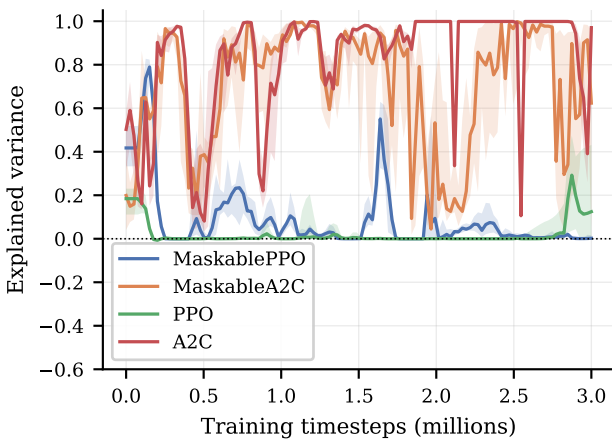
4.5 Training Diagnostics

By examining the explained variance, entropy and learning rate of each treatment, this section establishes both that the DRL treatments’ under-performance is not due to under-training and that the agents optimised successfully but their objective was wrong. No reward curve is reported, as SB3’s rolling `ep_raw_mean` buffer barely refreshes across 3M timesteps, and the null cells on the DQN family are a logging artefact rather than zeros: SB3 dumps on episode boundaries and DQN logs only 3 to 21 points, while throughput confirms the full budget completed.

Entropy comes first: A2C collapses to 0.07 ± 0.16 nats on physical, while on deeplearn it is pinned to the uniform ceiling ($\ln 513 = 6.24$) at 6.23 ± 0.00 across all seeds, with the worst critic of the three policy-gradient arms. Parameter-count asymmetry and the trace-reward interaction are both candidate causes.

Explained variance is the share of variance the critic accounts for, 1.0 being perfect and 0.0 no better than predicting the mean. Critic figures below are 2M-3M window averages. A2C’s near-deterministic policy (0.07 nats ≈ 1.07 effective actions of 513) with the best value-function fit in the study (0.944 ± 0.018 windowed) still does not beat a uniform draw over the same valid actions. This points to an optimised agent with a misspecified objective, not an under-trained one.

Critic fit does not translate into scheduling performance on physical: A2C’s fit gives no advantage over the random control, and MaskablePPO reaches the same result at 0.109 ± 0.263 . That poor fit follows from MaskablePPO’s $\sim 7,325$ gradient updates against A2C’s $\sim 600,000$, which is optimisation-budget asymmetry rather than one algorithm being better. Unmasked PPO makes the point from the other end, ending at exactly -0.000 ± 0.000 across all seeds

**Figure 6: Physical Trace - Explained Variance**

alongside a ~ 8.5 M episode reward. The expectation going in was that performance scales with the number of environment steps; these results show that reward shaping and optimisation budget matter considerably more.

5 DISCUSSION

This section discusses the results in Section 4 and how the findings respond to the research questions outlined in Section 1. DRL performance here is conditional on the workload: heuristics won on physical while MaskablePPO reaches parity-plus on deeplearn. What separates the two traces is headroom, the gap between the best heuristic and the random control. On physical that gap runs from LCFS (2,089.78 s) to the random control (2,166.19 s), a width of 3.7%, against 38.3% on deeplearn. There is roughly ten times more room on deeplearn for a learned policy to show a benefit. Three properties of the training setup bound how these results should be read. The reward defined in Section 3.2 emits only on every 512th action; at $\gamma = 0.99$ the next reward tick is worth $0.99^{512} = 0.0058$ against an effective horizon of 100 steps, so the signal being optimised is far sparser than the horizon it is discounted over. Second, Section 4.5 showed that the same 3M environment steps buy PPO 7,325 updates, A2C 600,000 and DQN 750,000, all under inherited SB3 defaults, so optimisation budget is not held constant across the families even though environment steps are. Third, with `ent_coef = 0.0` throughout, every policy-gradient treatment finishes concentrated (0.07-2.66 nats against the $\ln 513 = 6.24$ uniform ceiling), so under-exploration is not available as an explanation. Separating headroom from reward misalignment would need either an intermediate-heterogeneity trace at fixed reward or two identical traces compared under a denser reward.

5.0.1 RQ1: What does action masking contribute?

Masking acted asymmetrically. On physical the masked random control was inseparable from either masked treatment, while on deeplearn MaskablePPO beats that control by over 300 s. Nemenyi (Table 4) shows MaskablePPO separating from all four others but MaskableA2C failing to separate from its unmasked variant ($p = 0.090$), and the Page trend towards the masked treatment on physical is not significant on deeplearn. These do not point the same way, so only a narrow claim survives: masking supplies deployability, since an unmasked policy could never terminate an evaluation episode, but not protection against divergence, and MaskableDQN being evidence at $4,474 \pm 1,684$ s with a max slowdown of over 190k. It removes one failure mode, not all of them.

5.0.2 RQ2: What is the justification for the concentration on PPO?

A PPO-family treatment topped both traces but separated from the other leading treatments on neither, so no claim of better schedule quality is supported. With Nemenyi CD ($n = 10$, $k = 6$)

equal to 2.39 rank units, treatments falling within that range of each other are inconclusive, not equal. Cost is the axis where the family does hold an advantage: both PPO treatments trained 9-22% faster than the A2C variants and by a wider margin than the DQN family, which is what the update-budget asymmetry predicts. The question is whether a 9-22% margin on train time justifies a 40-45% concentration of the field.

5.0.3 RQ3: When is a DRL scheduler worth its inference cost?

These comparisons use heuristics run without backfill. On a single-resource workload heuristics are the clear choice, purely because there is so little space to optimise. On heterogeneous workloads the case is stronger, where a DRL treatment matches the heuristics on `avg_waiting` and beats all three on `avg_slowdown`. MaskablePPO was $34\times$ and $13\times$ slower than the fastest heuristics at evaluation time on physical and deeplearn. Out of sample the results then split in two directions. MaskablePPO pulls further ahead of LCFS ($-7.2\% \rightarrow -17.8\%$) and SJF ($-0.6\% \rightarrow -14.9\%$) while its lead over the random control collapses ($-27.7\% \rightarrow -3.9\%$). Both movements occur on the same split, so the widening margin over the heuristics is better read as a property of the holdout workload than as evidence of a better learned policy.

DRL on heterogeneous workloads can therefore only be recommended where fairness across all cases is the objective; where it is not, an unoptimised agent matching the heuristics on average waiting time does not justify the cost.

5.0.4 Threats to Validity

Every treatment optimises the same periodic negative mean queued waiting time, with no utilisation, fairness or makespan term. Applied identically to all six, it cannot explain the relative ranking between them, but it does bound every treatment's absolute ceiling, which is why the parity reported here should be read as a lower bound.

6 CONCLUSION & FUTURE WORK

This paper addressed the absence of systematic, statistically grounded validation in DRL-based HPC job scheduling, evaluating six DRL treatments, three Slurm heuristics and a masked uniform-random control over ten seeds on two real workload traces under a non-parametric, rank-based framework. For RQ1, masking contributes deployability rather than schedule quality: no unmasked policy terminated an evaluation episode on either trace, and while masking removes that failure mode it does not remove divergence, MaskableDQN being the clearest case. For RQ2, no PPO treatment separated from the leading A2C treatment on either trace, so neither supports a quality-based justification for the field's concentration on PPO; the measurable training-time advantage follows from an eightyfold lower gradient-update count under inherited defaults, a budget artefact rather than algorithmic efficiency. The one axis where a family does separate is inference latency, where the DQN family is fastest on both traces. RQ3 held on only one trace and only for tail fairness: on the homogeneous trace the agents could not beat a uniform draw over the same valid actions, and on the heterogeneous trace the advantage over the strongest heuristic is not measurable out of sample.

The strongest claim this study can make is therefore methodological. A masked random control, paired by seed, is inexpensive and decisive: without it, the parity the masked treatments reach with production heuristics on the physical trace

would have been reported as a success. It is recommended as a required baseline for this literature. These results are bounded by a reward proxy that emits once every 512 actions and by optimisation budgets differing by two orders of magnitude across the families, so the parity reported here is a lower bound. Establishing where that bound sits requires further study, beginning with the trace-gradient design outlined in Section 5. Once in place, exploration into reward function, budget optimisation and model distillation would examine ways in which a policy can be improved and costs can be reduced.

7 ACKNOWLEDGEMENT

The author thanks Prof. Clement Nyirenda for his supervision and assistance throughout the project. Computations were performed using the UWC's High Performance Computing and Research Cloud facilities. The authors acknowledge the use of OpenCode to assist with grammatical editing, sentence restructuring, and minor code refinement. All AI-assisted suggestions were reviewed, revised, and verified by the authors, who take full responsibility for the final content and code.

REFERENCES

- [1] Jacinto Carrasco, Salvador García, M Mar Rueda, Swagatam Das, and Francisco Herrera. 2020. Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review. *Swarm and Evolutionary Computation* 54, (2020), 100665.
- [2] Janez Demšar. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research* 7, (2006), 1–30.
- [3] Joaquín Derrac, Salvador García, Daniel Molina, and Francisco Herrera. 2011. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation* 1, 1 (2011), 3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>
- [4] Eelco Dolstra, Merijn de Jonge, and Eelco Visser. 2004. Nix: A Safe and Policy-Free System for Software Deployment. In *Proceedings of the 18th USENIX Large Installation System Administration Conference (LISA)*, 2004. 79–92.
- [5] Bradley Efron and Robert J Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall, New York.
- [6] Valerio Firmiano. 2024. Deep Reinforcement Learning for Dynamic Job-Shop Scheduling in High-Utilization Systems. Master's Thesis.
- [7] Xu Gao, Hang Dong, Lianji Zhang, Yibo Wang, Xianliang Yang, and Zhenyu Li. 2025. Self-Attention Mechanisms in HPC Job Scheduling: A Novel Framework Combining Gated Transformers and Enhanced PPO. *Applied Sciences (Switzerland)* 15, 16 (August 2025). <https://doi.org/10.3390/app15168928>
- [8] I. Gitler, C. J. Barrios Hernández, and E. Meneses. 2022. *High Performance Computing: 8th Latin American Conference, CARLA 2021, Guadalajara, Mexico, October 6–8, 2021, Revised Selected Papers*. Springer International Publishing.
- [9] Yan Gu, Zhaoze Liu, Shuhong Dai, Cong Liu, Ying Wang, Shen Wang, Georgios Theodoropoulos, and Long Cheng. 2025. Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review. Retrieved from <http://arxiv.org/abs/2501.01007>
- [10] G. J. Henry. 1984. The UNIX System: The Fair Share Scheduler. *AT&T Bell Laboratories Technical Journal* 63, 8 (October 1984), 1845–1857. <https://doi.org/10.1002/j.1538-7305.1984.tb00068.x>
- [11] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.
- [12] Daniël Lakens. 2017. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8, 4 (2017), 355–362.
- [13] J. R. Landis and G. G. Koch. 1977. "An Application of Hierarchical Kappa-type Statistics in the Assessment of Majority Agreement among Multiple Observers". *Biometrics* 33, 2 (June 1977), 363. <https://doi.org/10.2307/2529786>

- [14] Erich Leo Lehmann and Howard JM D'Abrera. 2006. *Nonparametrics: statistical methods based on ranks*. Springer New York.
- [15] David A. Lifka. 1995. The ANL/IBM SP Scheduling System. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP'95) (Lecture Notes in Computer Science)*, 1995. Springer, 295–303. https://doi.org/10.1007/3-540-60153-8_35
- [16] S. Sudheer Mangalampalli, Ganesh Reddy Karri, Sachi Nandan Mohanty, Shahid Ali, Muhammad Ijaz Khan, Sherzod Abdullaev, and Salman A. Alqahtani. 2024. Multi-Objective Prioritized Task Scheduler Using Improved Asynchronous Advantage Actor Critic (a3c) Algorithm in Multi Cloud Environment. *IEEE Access* 12, (2024), 11354–11377. <https://doi.org/10.1109/ACCESS.2024.3355092>
- [17] Fady Nashat Manhary, Marghny H. Mohamed, and Mamdouh Farouk. 2025. A scalable machine learning strategy for resource allocation in database. *Scientific Reports* 15, 1 (December 2025). <https://doi.org/10.1038/s41598-025-14962-5>
- [18] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource Management with Deep Reinforcement Learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016. ACM, 50–56.
- [19] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning Scheduling Algorithms for Data Processing Clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019. ACM, 270–288.
- [20] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Tim Harley, Timothy P. Lillicrap, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016) (Proceedings of Machine Learning Research)*, 2016. PMLR, 1928–1937. Retrieved from <http://proceedings.mlr.press/v48/mnih16.html>
- [21] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (February 2015), 529–533. <https://doi.org/10.1038/nature14236>
- [22] F Mölder, KP Jablonski, B Letcher, MB Hall, PC van Dyken, CH Tomkins-Tinch, V Sochat, J Forster, FG Vieira, C Meesters, S Lee, SO Twardziok, A Kunitz, J VanCampen, V Malladi, A Wilm, M Holtgrewe, S Rahmann, S Nahnsen, and J Köster. 2025. Sustainable data analysis with Snakemake [version 3; peer review: 2 approved]. *F1000Research* 10, 33 (2025). <https://doi.org/10.12688/f1000research.29032.3>
- [23] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8. Retrieved from <http://jmlr.org/papers/v22/20-1364.html>
- [24] Yousef Sanjalawe, Salam Al-E'mari, Salam Fraihat, and Sharif Makhadmeh. 2025. AI-driven job scheduling in cloud computing: a comprehensive review. *Artificial Intelligence Review* 58, 7 (July 2025). <https://doi.org/10.1007/s10462-025-11208-8>
- [25] Prasanna Sankaran, Shiva Kiran Lingishetty, and Mrinal Kumar. 2025. Leveraging Reinforcement Learning for Efficient Task Scheduling in Multi-Cloud Environments. *International Journal of Innovative Research in Computer Science and Technology* 13, 2 (April 2025), 35–41. <https://doi.org/10.55524/ijircst.2025.13.2.6>
- [26] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. Retrieved from <http://arxiv.org/abs/1707.06347>
- [27] Aasish Kumar Sharma and Julian Kunkel. 2025. GrapheonRL: A Graph Neural Network and Reinforcement Learning Framework for Constraint and Data-Aware Workflow Mapping and Scheduling in Heterogeneous HPC Systems. Retrieved from <http://arxiv.org/abs/2506.00260>
- [28] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. 2018. *Operating System Concepts* (10th ed.). Wiley, Hoboken, NJ.
- [29] Roland Stolz, Hanna Krasowski, Jakob Thumm, Michael Eichelbeck, Philipp Gassert, and Matthias Althoff. 2024. Excluding the Irrelevant: Focusing Reinforcement Learning through Continuous Action Masking. In *Advances in Neural Information Processing Systems*, 2024. Curran Associates, Inc., 95067–95094. <https://doi.org/10.52202/079017-3013>
- [30] R.S. Sutton and A.G. Barto. 2018. *Reinforcement Learning, second edition: An Introduction*. MIT Press. Retrieved from <https://books.google.co.za/books?id=uWV0DwAAQBAJ>
- [31] Top500. 2026. TOP500 List Statistics.
- [32] Hao Wang, Xu Chen, Kunpeng Zhang, and Robert H Smith. 2024. *Dynamic Multi-objective Flexible Job Shop Scheduling: A Three-phase Deep Reinforcement Learning Approach*. Retrieved from <https://ssrn.com/abstract=5312558>
- [33] Lingfei Wang, Maria A. Rodriguez, and Nir Lipovetzky. 2025. MetaPilot: A DRL-based controller for dynamic adaptation to shifting scheduling objectives in HPC systems. *Future Generation Computer Systems* 178, (May 2025), 108269. <https://doi.org/10.1016/j.future.2025.108269>
- [34] Lingfei Wang, Maria A. Rodriguez, and Nir Lipovetzky. 2025. Optimizing HPC scheduling: a hierarchical reinforcement learning approach for intelligent job selection and allocation. *Journal of Supercomputing* 81, 8 (June 2025). <https://doi.org/10.1007/s11227-025-07396-3>
- [35] Rui Wu, Jianxin Zheng, and Xiyan Yin. 2025. Dynamic Scheduling for Multi-Objective Flexible Job Shops with Machine Breakdown by Deep Reinforcement Learning. *Processes* 13, 4 (April 2025). <https://doi.org/10.3390/pr13041246>
- [36] Andy B. Yoo, Morris A. Jette, and Mark Grondona. 2003. SLURM: Simple Linux Utility for Resource Management. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2003) (Lecture Notes in Computer Science)*, 2003. Springer, 44–60. https://doi.org/10.1007/10968987_3
- [37] Di Zhang, Dong Dai, Youbiao He, Forrest Sheng Bao, and Bing Xie. 2020. RLScheduler: An Automated HPC Batch Job Scheduler Using Reinforcement Learning. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. <https://doi.org/10.1109/SC41405.2020.00035>